VIETNAM NATIONAL UNIVERSITY, HANOI

INTERNATIONAL SCHOOL



FINAL PROJECT

TOPIC: STRING MATCHING ALGORITHMS AND APPLICATIONS: Z-FUNCTION, KNUTH-MORRIS-PRATT, AND RABIN-KARP

Class: INS3050 - INS305001

Lecturer: Ta Dinh Quy, Msc

Group 7

Student name		Contribution
Mạc Hải Nam		26,5%
Dương Thu Phượng		24,5%
Hà Phương Thảo		24,5%
Nguyễn Thanh Tùng		24,5%

Ha Noi, August 2026

WORK ALLOCATION AND PROJECT TIMELINE

Group 7 - INS3050 - String Matching Algorithms and Applications

Member	Assigned task	Deliverable
Phase 1 - Introduction (midterm)		
Hà Phương Thảo	Problem definition and foundational data structures	Sections 1.1 and 1.2
Mạc Hải Nam	Real-world applications and citations	Section 1.5 and the reference list
Nguyễn Thanh Tùng	Report outline and scope	Section 1.6 and chapter structure
Dương Thu Phượng	Cover page and integration	Compiled midterm submission
Phase 2 - Chapter 2: Algorithmic Analysis		
Hà Phương Thảo	Z-function and KMP: mechanics and complexity proofs	Sections 2.1 and 2.2
Mạc Hải Nam	Rabin-Karp: rolling hash and complexity	Section 2.3
Nguyễn Thanh Tùng	Brute-Force baseline and theoretical comparison	Section 2.4
Dương Thu Phượng	Cross-verification of complexity proofs and notation	Verified Chapter 2 and the complexity table
Phase 3 - Chapter 3: Implementation and Benchmarking		
Nguyễn Thanh Tùng	Write-up of the implementation and correctness procedure	Section 3.1
Hà Phương Thảo	Write-up of the methodology and datasets	Section 3.2
Mạc Hải Nam	Full implementation, benchmarking and all figures	Source code and Section 3.3
Dương Thu Phượng	Analysis of the empirical results and conclusions	Sections 3.4, Chapter 4, Slides and Write report

Table of Contents

CHAPTER 1: INTRODUCTION TO THE PROBLEM 1

1.1. Formal Problem Description 1

1.2. Foundational Data Structures and Core Concepts 2

1.3. Baseline Approach: Brute-Force 3

1.4. Technical Constraints and Challenges 3

1.5. Real-World Applications 5

1.6. Scope and Approach 6

CHAPTER 2: ALGORITHMIC ANALYSIS 7

2.1. Z-function: LCP Construction and Matching 7

2.2. KMP: Prefix Function and Backtrack-free Matching 8

2.3. Rabin-Karp: Rolling Hash and Randomized Matching 9

2.4. Brute-Force Baseline and Theoretical Comparison 11

CHAPTER 3: IMPLEMENTATION AND BENCHMARKING 14

3.1. Implementation Description 14

3.2. Experimental Methodology and Dataset Preparation 15

3.3. Empirical Analysis and Performance Visualization 16

3.4. Empirical Discussion 21

CHAPTER 4: CONCLUSION 24

REFERENCES 25

LIST OF FIGURES

Figure 3.1: Execution time against text length n on log-log axes, one panel per corpus (m = 50) 17
Figure 3.2: Search time by corpus at n = 10^6 and m = 50. Alphabet sizes are 2 for binary and periodic, 4 for dna and 96 for English 18
Figure 3.3: Ratio of Brute-Force to KMP against pattern length at n = 10^5. Left: wall-clock time. Right: character comparisons 19
Figure 3.4: Rabin-Karp search time and false-positive count against modulus q, n = 10^6 20

LIST OF TABLES

Table 1.1: Technical Constraints Summary 4
Table 2.1: Prefix function of "abacaba" 8
Table 2.2: Theoretical comparison of the four algorithms 13
Table 3.1: The four benchmark corpora 15
Table 3.2: Mean search time in seconds at m = 50, at the two largest text sizes. Smaller sizes are plotted in Figure 3.1 17
Table 3.3: Ratio of Brute-Force to KMP character comparisons at n = 10^5, by pattern length 19
Table 3.4: Rabin-Karp mean time in seconds and mean false-positive count against modulus, n = 10^6, m = 50 20
Table 3.5: Single-run search time in seconds against the built-in implementation, m = 50 21

CHAPTER 1: INTRODUCTION TO THE PROBLEM

1.1. Formal Problem Description

The String Matching problem is a fundamental challenge in computer science: locating all instances of a specific short string (Pattern P) inside a longer text string (Text T).

In everyday terms, string matching is what happens every time you press Ctrl+F in a document and type a word: the computer must find every place that word appears in the text. The word you are searching for is called the pattern (P), and the document you search through is called the text (T). The task is to report every position in T where P appears exactly. Two details make the problem less trivial than it first looks: the same match may overlap with another (searching "aba" in "ababa" yields matches at both position 0 and position 2, sharing the middle "a"), and we must find all occurrences, not merely the first. The formal definition below states this precisely.

Mathematical Definition:

The problem is defined over an alphabet Σ, a finite set of characters such as {A, C, G, T} for DNA sequences or Unicode for natural languages. The text T is a string of length n, represented as T[0..n-1], while the pattern P is a string of length m, represented as P[0..m-1], subject to the constraint $m \leq n$. The output is the set of all valid starting indices i, where $0 \leq i \leq n-m$, at which the substring of T matches P exactly, that is Output = $\{i \in N \mid 0 \leq i \leq n-m \text{ and } T[i..i+m-1] = P[0..m-1]\}$.

Concrete Example:

Consider the text T = "mississippi" of length n = 11, indexed from 0 to 10, together with the pattern P = "issi" of length m = 4, indexed from 0 to 3. At index i = 1 the substring T[1..4] = "issi" is equal to P, which gives a valid match, and the same occurs at index i = 4 where T[4..7] = "issi". The algorithm output is therefore Output = {1, 4}.

Input/Output and Sliding Mechanism Diagram:

```
INPUT:
  Text (T)    : | m | i | s | s | i | s | s | i | p | p | i |   (n = 11)
  Index       :   0   1   2   3   4   5   6   7   8   9  10
  Pattern (P) : | i | s | s | i |   (m = 4)


SLIDING MECHANISM:
  i = 0:  T: [m i s s] i s s i p p i
          P:  i s s i           --> MISMATCH ('m' != 'i')
  i = 1:  T: m [i s s i] s s i p p i
          P:    i s s i         --> EXACT MATCH (Index 1)
  i = 2:  T: m i [s s i s] s i p p i
          P:      i s s i       --> MISMATCH ('s' != 'i')
  i = 3:  T: m i s [s i s s] i p p i
```

```
          P:        i s s i      --> MISMATCH ('s' != 'i')
  i = 4:  T: m i s s [i s s i] p p i
          P:          i s s i    --> EXACT MATCH (Index 4)


OUTPUT:
  Set of matching starting indices: {1, 4}
```

1.2. Foundational Data Structures and Core Concepts

To optimize string matching beyond naive character scanning, the algorithms studied in this report preprocess strings into specialized structures based on three core concepts.

1.2.1. Prefix, Suffix, and the Concept of a Border (Foundation of KMP)

A prefix is a substring of S starting at the first character, denoted S[0..k] where $0 \leq k < |S|$; a proper prefix excludes the string itself. Symmetrically, a suffix is a substring of S ending at the last character, denoted S[k..|S|-1] where $0 \leq k < |S|$, and a proper suffix likewise excludes the string itself. A border is a substring that is simultaneously a proper prefix and a proper suffix of S. For the string S = "abacaba", for instance, the substring "aba" is a border because it both starts and ends the string. The KMP algorithm uses this concept, via the π-array, to skip redundant comparisons by shifting the pattern safely without moving the text pointer backward (Knuth, Morris, & Pratt, 1977).

1.2.2. The Z-array Structure (Foundation of Z-function)

The Z-array is computed for a string S of length n. For any index i ($1 \leq i < n$), Z[i] stores the length of the Longest Common Prefix (LCP) between the entire string S and its suffix starting at index i. By convention Z[0] is either set to N or left undefined, since the whole string trivially matches itself (Gusfield, 1997).

For the string S = "abacaba", Z[1] = 0 because the suffix "bacaba" begins with 'b', which does not match the prefix character 'a'. Z[2] = 1 because the suffix "acaba" shares only the first character 'a' with the prefix, the second characters ('c' versus 'b') differing. Z[4] = 3 because the suffix "aba" matches the first three characters of S exactly. The resulting array is therefore Z = [-, 0, 1, 0, 3, 0, 1]. This structure enables linear-time pattern matching by evaluating the concatenated string P + "$" + T, where "$" is a separator character that occurs in neither P nor T.

1.2.3. Rolling Hash (Foundation of Rabin-Karp)

A Rolling Hash is a hashing technique designed to compute the hash value of a sliding window in constant O(1) time. Instead of recalculating the hash from scratch over O(m) steps, it updates the previous hash value mathematically by removing the character leaving the window and adding the character entering it.

The polynomial rolling hash is defined as $H(S) = (\Sigma S[i] \cdot p^{(m-1-i)}) \bmod q$, for i = 0 to m-1, where p is a base at least as large as the alphabet and q is a large prime modulus (Karp & Rabin, 1987).

1.3. Baseline Approach: Brute-Force

The Brute-Force (Exhaustive Search) approach serves as the foundational baseline to measure the efficiency of optimized algorithms.

Algorithmic Idea:

The algorithm aligns the pattern P at every possible shift index i from 0 to n-m within the text T. An inner loop compares each character sequentially (T[i+j] vs P[j] for $0 \leq j < m$). If a mismatch occurs, the inner loop aborts immediately, the pattern shifts right by exactly one position (i = i+1), and the comparison resets entirely from P[0].

Complexity Analysis:

In the best case the algorithm runs in O(n) time, which occurs when the first character of P rarely appears in T and the inner loop therefore terminates immediately. In the worst case it runs in O(n×m) time, which occurs when the inner loop must scan almost all m characters before failing - a situation commonly seen in highly repetitive text such as T = "aaaa...aaa" with P = "aaaa". The space complexity is O(1), since the method runs completely in place with no auxiliary storage or preprocessing overhead.

1.4. Technical Constraints and Challenges

Production-grade string matching must overcome severe engineering and algorithmic bottlenecks determined by data characteristics.

1.4.1. Massive Data Scale

In modern environments like Bioinformatics and Big Data, text streams are massive. For instance, the Human Genome consists of approximately 3×10^9 base pairs. Using a Brute-Force approach with a complexity of O(n×m) is computationally prohibitive, making a near-linear processing time of O(n+m) an absolute requirement (Cormen, Leiserson, Rivest, & Stein, 2009). The same constraint drives sequence-alignment tools such as BLAST, which rely on fast exact matching of short seeds before any approximate alignment is attempted (Altschul, Gish, Miller, Myers, & Lipman, 1990).

1.4.2. Alphabet Size Variations (Σ)

In small alphabets, such as DNA strings where $|\Sigma| = 4$, character repetitions are highly frequent. Brute-Force struggles here because it constantly triggers near-matches that fail late, whereas algorithms such as KMP and the Z-function thrive on these repetitive structures by utilizing prefixes and borders to skip text segments.

In large alphabets, such as Extended Unicode, random character alignments are rare, so a naive scan usually fails on the first character and the practical cost of every algorithm falls. The remaining concern is Rabin-Karp: the polynomial hash requires a base at least as large as $\|\Sigma\|$, and a wider character range demands a correspondingly large modulus if the hash is to stay evenly distributed.

1.4.3. Periodic and Degenerate Inputs

Highly repetitive text sequences - such as a^n or (ab)^(n/2) - trigger the worst-case scenarios for naive string matching. Without structural memory or lookup arrays, pointers are forced to

backtrack repeatedly, severely reducing execution efficiency.

1.4.4. Hash Collisions in Rabin-Karp

Because hash values map into a finite integer space (32-bit or 64-bit), distinct substrings can yield identical hash values, causing False Positives.

Choosing a prime modulus q that is too small triggers frequent collisions, forcing character-by-character validation checks that can degrade performance toward $O(n \times m)$. Choosing an excessively large q, on the other hand, introduces heavy big-integer arithmetic overhead on every window of the text. Which of these two costs dominates is an empirical question, and Chapter 3 measures it directly by sweeping q across five orders of magnitude.

1.4.5. Algorithm Suitability, Strengths, and Weaknesses

The constraints above explain why no single algorithm is best for every input. Each algorithm is tuned to a different data profile, and each buys its speed with a specific trade-off. The following summary connects every algorithm to the kind of data it suits and states its strengths and weaknesses explicitly.

Brute-Force needs no preprocessing and no memory, but re-compares characters it has already seen and degrades to $O(n \times m)$ on repetitive data. KMP guarantees $\Theta(n+m)$ at a memory cost of only $\Theta(m)$ and never moves the text pointer backward, which is what allows it to run on streams, but it cannot skip characters. The Z-function offers the same guarantee and is the easiest of the three to prove correct, at the price of $\Theta(n+m)$ auxiliary space because its array spans the whole concatenated string. Rabin-Karp needs only $O(1)$ space and is the only one of the four that extends naturally to searching many patterns at once, but its cost depends entirely on the quality of the hash: too small a modulus produces collisions that force character-by-character verification and push the worst case back to $O(n \times m)$.

Section 2.4 develops these trade-offs into a full comparison across preprocessing cost, best, average and worst-case time, and auxiliary space.

Table 1.1: Technical Constraints Summary

Technical Parameter	Practical Scale / Context	Most Affected Algorithm	Core Mitigation Strategy
Massive Text Size (n)	10^6 to 10^9 characters (Genomics / Logs)	Brute-Force	Transition to near-linear time $O(n+m)$ architectures.
Small Alphabet Size ($\lvert\Sigma\rvert \leq 4$)	DNA bases {A,C,G,T}, binary streams	Brute-Force	Deploy structural preprocessing functions (KMP, Z-array).
Large Alphabet Size	Global Unicode, international lexicons	Rabin-Karp (hash base and range)	Choose a base at least as large as $\lVert\Sigma\rVert$ and a modulus that keeps the hash evenly distributed.

Periodic / Degenerate Strings	Inputs structured as a^n or (ab)^(n/2)	Brute-Force	Apply smart border shifting to eliminate pointer backtracking.
Hash Collision Vulnerability	Inversely proportional to modulus q	Rabin-Karp	Implement double hashing and strict character verification fallbacks.

1.5. Real-World Applications

String matching is not an abstract exercise: it is the computational core of systems people use every day. To show concretely how the three algorithms studied in this report behave, this section takes a single representative application - in-file text search, the operation behind the Ctrl+F command and the Unix utility grep - and applies KMP, the Z-function, and Rabin-Karp to it in turn (Manning, Raghavan, & Schütze, 2008; Jurafsky & Martin, 2009). In this setting a fixed pattern P must be located inside a much longer text T, and every occurrence must be reported - exactly the problem all three algorithms solve, each in its own way. Full pseudocode for each algorithm is given in Chapter 2 and the working Python implementations are described in Chapter 3; this section concerns only how each behaves in this particular application.

1.5.1. Applying KMP

KMP first builds the π-array of the pattern, then scans the text once without ever moving the text pointer backward. This makes it the natural choice when the text arrives as a stream or is too large to store in full.

1.5.2. Applying the Z-function

The Z-function concatenates the pattern, a separator, and the text, then computes one Z-array over the whole string. Every position whose Z-value equals the pattern length marks a match. It is the simplest of the three to reason about when the entire text fits in memory.

1.5.3. Applying Rabin-Karp

Rabin-Karp turns each window of the text into a hash value and slides that hash in constant time, comparing characters only when two hashes agree. This is what makes it extend so naturally to searching for many patterns at once.

1.5.4. Which Algorithm Fits This Application

For searching ordinary text on a large alphabet, all three run in linear time, but their differences from Section 1.4.5 decide the best fit. KMP is preferred when the text is streamed or memory is tight, since it needs only O(m) extra space. The Z-function is the clearest to implement when the whole file is already in memory. Rabin-Karp becomes the natural choice the moment the task grows from one pattern to many - for example scanning a document against a list of forbidden phrases - because a single hash table can hold every pattern at once; the classical alternative for that setting is the Aho-Corasick automaton (Aho & Corasick, 1975). Chapter 3 will benchmark these three implementations, against the Brute-Force baseline, to confirm these expectations empirically.

1.6. Scope and Approach

The central premise of this report is that the algorithms studied are not interchangeable. Each trades preprocessing cost, memory, and worst-case guarantees differently, and the appropriate choice depends on measurable properties of the input - the text length n, the pattern length m, the alphabet size $|\Sigma|$, and the periodicity of the text.

Rather than accepting textbook Big-O values at face value, this project implements all four algorithms under identical conditions and measures them on the same data. The purpose of the measurement is not to declare a single winner but to establish which property of the input governs each algorithm's cost, and to determine whether the constant factors hidden inside asymptotic notation are large enough to change the ranking in practice. Chapter 2 develops the theory that generates these expectations, and Chapter 3 tests them against four corpora spanning alphabet sizes from 2 to 96 and text structures from natural language to constructed periodic strings.

CHAPTER 2: ALGORITHMIC ANALYSIS

2.1. Z-function: LCP Construction and Matching

2.1.1. Definition of the Z-function

For a string S of length n, the Z-array is defined so that Z[i] is the length of the longest common prefix between the whole string S and the suffix of S that starts at position i. The value Z[0] is conventionally written as 0 or left undefined, since the whole string trivially matches itself.

For S = "abacaba" the Z-array is [-, 0, 1, 0, 3, 0, 1]. For instance Z[4] = 3, because the suffix "aba" beginning at position 4 shares its first three characters with the prefix of S; and Z[1] = 0, because the suffix "bacaba" begins with 'b', which does not match the prefix character 'a'.

2.1.2. The Z-box [l, r)

The linear-time construction maintains a Z-box [l, r), meaning that S[l..r-1] is known to match the prefix S[0..r-l-1]; the right endpoint r is exclusive. This interval stores matching information that can be reused instead of comparing characters from the beginning at every position. Three cases arise.

Case 1 - i lies outside the current Z-box ($i \geq r$). No previous information can be reused. Set l = r = i and compare S[r] against S[r-l] while the characters match. Each successful comparison extends r. When comparison stops, Z[i] = r - l.

Case 2 - i lies inside the Z-box ($i < r$) and the copied value does not reach the right boundary. Let k = i - l. If $Z[k] < r - i$, the known prefix match fits entirely inside the current box, so Z[i] = Z[k] and no character comparison is needed at all.

Case 3 - i lies inside the Z-box and the copied value reaches the right boundary. If $Z[k] \geq r - i$, the stored information is insufficient to determine Z[i]. Set l = i and continue comparing characters from the right boundary onwards, extending the Z-box where possible.

The implementation in Chapter 3 collapses these three cases into a single expression, ZS[i] = min(r - i, Z[i - l]) followed by a while loop that extends the value. The two formulations are equivalent: the min takes the copied value when it fits inside the box and truncates it at the boundary otherwise, and the while loop performs the additional comparisons required by Case 1 and Case 3.

2.1.3. Θ(n + m) Analysis and Pattern Matching

The key amortised argument is that r never moves backwards. Every successful comparison inside a while loop increases r by at least one, and r can only advance as far as the end of the string, so the total number of successful character comparisons over the entire algorithm is O(n). Each while loop terminates with at most one unsuccessful comparison, so failures are also O(n). All remaining work per position is constant. Computing the Z-array therefore takes Θ(n).

For pattern matching the pattern and text are combined into S = P + separator + T. With |P| = m and |T| = n the combined string has length n + m + 1, so the complete matching procedure runs in Θ(n + m). Every position whose Z-value equals m marks an occurrence, and the corresponding position in T is i - m - 1.

The separator must not occur in either P or T. Otherwise a prefix match could cross the boundary between the two parts and make a Z-value look like a complete pattern occurrence when it is not.

The group uses the byte \x00 rather than the textbook symbol $, because real English text can legitimately contain a dollar sign whereas the null byte is absent from all four corpora used in the benchmark.

The cost of this construction is memory. The Z-array spans the entire concatenation and therefore requires $\Theta(n + m)$ auxiliary space, against $\Theta(m)$ for KMP. Section 3.3 shows the practical consequence: the Z-function trails KMP by a factor of between 2.3 and 2.8 at $n = 10^6$ on every corpus despite sharing the same asymptotic bound.

2.2. KMP: Prefix Function and Backtrack-free Matching

2.2.1. Border and the Prefix Function π

For a string X, a border is a proper prefix of X that is also a suffix of X. The KMP prefix function $\pi[i]$ is the length of the longest border of the prefix S[0..i]. It records how much of a previous partial match can be reused after a mismatch.

Table 2.1: Prefix function of "abacaba"

i	0	1	2	3	4	5	6
S[i]	a	b	a	c	a	b	a
π[i]	0	0	1	0	1	2	3

At i = 2 the border is "a", of length 1. At i = 3 no non-empty border exists, so $\pi[3] = 0$. At i = 6 the longest border is "aba", so $\pi[6] = 3$.

2.2.2. Reusing Previous Matches

Suppose the pattern has already matched through position j - 1 and the next character mismatches. A naive algorithm restarts the pattern from its first character, discarding everything it has learned. KMP instead sets $j = \pi[j-1]$. The prefix function states exactly how much of the already matched portion is also a prefix of the pattern, so the pattern can be realigned to that point without re-examining any text character.

This is the central idea of KMP: the pattern moves logically forward while the text pointer stays where it is. The information encoded in π eliminates redundant comparisons without discarding any possible match. When a full match is found the same rule is applied, $j = \pi[m-1]$, which is what allows overlapping occurrences to be reported - searching "aba" in "ababa" correctly yields both position 0 and position 2.

2.2.3. Θ(n + m) Amortised Analysis

Let i be the text pointer and j the pattern pointer. During the search i only increases. When T[i] = P[j] both pointers advance, so j increases by exactly one. When a mismatch occurs with $j > 0$, KMP replaces j by $\pi[j-1]$, which decreases j.

The total amount by which j increases over the whole search is at most n, because each increase is paired with an advance of i and i never exceeds n. Since j is bounded below by zero, the total amount of decrease cannot exceed the total amount of increase. All fallback operations together

are therefore O(n), even though a single mismatch may trigger several consecutive decreases.

The construction of π has the same amortised structure: its border length can increase at most m times in total, and every decrease is charged against an earlier increase, giving Θ(m). Searching takes Θ(n), so the combined algorithm runs in Θ(n + m) in every case - best, average and worst alike. Auxiliary space is Θ(m), the π-array of the pattern only.

2.2.4. The Streaming Property

Because the text pointer never moves backwards, KMP can process a stream of incoming data without storing the complete text. It needs only the current pattern position, the prefix-function array and the current input character. This matters whenever the text is too large to hold in memory or arrives continuously, as in network intrusion detection, and it is the property that distinguishes KMP from the Z-function in practice: the Z-function requires the entire concatenation to be materialised before it can begin.

2.3. Rabin-Karp: Rolling Hash and Randomized Matching

The three algorithms presented so far all compare characters directly. Rabin-Karp takes a different route: it reduces every window of the text to a single integer and compares integers instead of strings. An integer comparison costs O(1) regardless of the pattern length, whereas a string comparison costs O(m). The difficulty is that computing the integer for each window from scratch would itself cost O(m), which would give away the entire advantage. The rolling hash solves exactly this problem.

2.3.1. The Polynomial Hash Function

A string is treated as a number written in a positional numeral system. For a string S of length m over an alphabet Σ, the polynomial hash is defined as

$$H(S) = (S[0]\cdot b^{(m-1)} + S[1]\cdot b^{(m-2)} + \ldots + S[m-1]\cdot b^{0}) \bmod q$$

where b is the base and q is a large prime modulus. Two constraints govern the choice of these constants. The base b must be at least as large as the alphabet, otherwise two distinct characters would occupy overlapping ranges and collisions would become systematic rather than accidental; this implementation uses b = 256, which covers any single-byte alphabet. The modulus q keeps the value inside machine word range, and its size controls how often distinct strings receive the same hash. Section 3.3 measures this effect directly across five values of q.

2.3.2. The Rolling Update

The central idea is that the hash of the window starting at position i+1 can be derived from the hash of the window starting at position i in constant time. Three operations are required: remove the character leaving the window, shift the remaining characters up one positional place, and add the character entering the window.

Writing H(i) for the hash of the window T[i..i+m-1], the update is

$$H(i+1) = ((H(i) - T[i]\cdot b^{(m-1)}) \cdot b + T[i+m]) \bmod q$$

The subtracted term removes the contribution of the departing character, which carries the highest positional weight b^(m-1). Multiplying by b promotes every remaining character one place.

Adding the incoming character gives it weight b^0. Because b^(m-1) mod q does not depend on i, it is computed once during preprocessing and reused for every window.

The mechanism is easier to see in base ten. Taking a three-digit window over the digit string 1234, the window 123 becomes 234 by the arithmetic $(123 - 1\times100)\times10 + 4 = 23\times10 + 4 = 234$. The rolling hash performs precisely this operation, with b in place of 10 and a modulus applied at each step to prevent overflow.

One implementation detail deserves note. Because the subtraction is performed before the modulus is reapplied, the intermediate value can become negative in languages whose remainder operator follows the sign of the dividend. In such languages the result must be corrected by adding q whenever it falls below zero. Python's modulo operator always returns a non-negative result for a positive modulus, so no correction is needed here - but the same code transliterated to C or Java would be incorrect without it.

2.3.3. Verification and False Positives

A hash maps a space of |Σ|^m possible strings onto q possible values. Since q is finite and far smaller than |Σ|^m, distinct strings must sometimes share a hash value. Equality of hashes is therefore a necessary but not a sufficient condition for a match: unequal hashes guarantee that the strings differ, while equal hashes only make a match plausible.

Every hash agreement must consequently be confirmed by an explicit character comparison. When the confirmation fails, the event is called a false positive, and the O(m) spent on it is wasted work. The frequency of false positives is governed by q, and the trade-off is not one-sided: a small q produces frequent collisions and therefore frequent verification, while a large q reduces collisions but makes the modular arithmetic on every window more expensive. Which of the two costs dominates is an empirical question rather than a theoretical one, and Section 3.3 answers it by measurement.

2.3.4. Pseudocode

```
RABIN-KARP(T, P, b, q)
  n <- length(T);  m <- length(P)
  if m > n then return empty list
  high <- b^(m-1) mod q
  ph <- 0;  th <- 0
  for i <- 0 to m - 1
      ph <- (ph * b + code(P[i])) mod q
      th <- (th * b + code(T[i])) mod q
  result <- empty list
  for i <- 0 to n - m
      if ph = th
          if T[i .. i + m - 1] = P then append i to result
      if i < n - m
          th <- ((th - code(T[i]) * high) * b + code(T[i + m])) mod q
```

```
return result
```

2.3.5. Complexity Analysis

Preprocessing consists of computing b^(m-1) mod q and the two initial hashes, which is Θ(m). The main loop executes n - m + 1 times, and each iteration performs a constant number of arithmetic operations, giving Θ(n) for the scan itself. The total cost is therefore Θ(n + m) plus the cost of verification.

In the average case, under the assumption that the hash distributes values uniformly, the expected number of spurious agreements is approximately n/q. When q is chosen large relative to n, this term is negligible and the algorithm runs in O(n + m) overall.

In the worst case every window agrees with the pattern hash and every agreement requires an O(m) verification, degrading the algorithm to O(n×m) - the same bound as Brute-Force. This is not merely a theoretical concern: it occurs whenever the modulus is too small for the text length, and Section 3.3 exhibits a case in which 499,975 of roughly one million windows collide.

Auxiliary space is O(1). Unlike KMP, which stores a π-array of length m, and unlike the Z-function, which stores an array over the whole concatenation, Rabin-Karp holds only two integers and a precomputed weight. This makes it the cheapest of the four in memory. The same constant-space property underlies its main practical advantage: hashes for many patterns of equal length can share a single hash table, so k patterns are found in O(n + km) rather than the O(k(n + m)) that repeated runs of KMP would cost.

2.4. Brute-Force Baseline and Theoretical Comparison

The three advanced algorithms examined above all introduce mechanisms for bypassing redundant comparisons. Brute-Force takes the most direct route: it aligns the pattern against the text and compares characters one by one from left to right. On a mismatch it abandons the current alignment, shifts the pattern exactly one position to the right, and restarts comparison from the first character of the pattern. Naive and inefficient though it is in the worst case, it serves as the baseline against which every other algorithm in this report is measured.

2.4.1. The Alignment and Comparison Process

Given a text T of length n and a pattern P of length m, the algorithm evaluates every possible starting position i, where $0 \leq i \leq n - m$. At each position it initialises an inner index j at 0 and compares T[i+j] against P[j]. If all m characters match, position i is recorded as an occurrence. The moment a single pair fails to match, the inner loop terminates and the outer loop advances the alignment by one.

2.4.2. Redundancy and the Worst Case

The weakness of the approach lies in its complete disregard for information gathered during previous comparisons. When a mismatch occurs after j successful matches, the algorithm discards the knowledge that those j characters of the text are already known, shifts by a single position, and re-compares overlapping material.

This creates a severe worst case. Consider a text consisting of a single repeated character, T = "AAAAAAAAAA", against a pattern differing only at its final position, P = "AAAAB". At every

starting position the algorithm compares the first m - 1 characters successfully before failing on the last, forcing close to m operations at nearly every offset and driving the total to scale with the product of the two lengths.

The same effect appears on the periodic corpus used in Chapter 3, where the text has period 2 and the pattern is sampled from the text itself. Section 3.3 measures it: the ratio of Brute-Force comparisons to KMP comparisons rises from 3.00 at m = 5 to 248.01 at m = 500, approximately m/2, while remaining close to 1.0 on the three non-periodic corpora.

2.4.3. Pseudocode

```
BRUTE-FORCE(T, P)
  n <- length(T);  m <- length(P)
  if m > n then return empty list
  result <- empty list
  for i <- 0 to n - m
      j <- 0
      while j < m and T[i + j] = P[j]
          j <- j + 1
      if j = m
          append i to result
  return result
```

2.4.4. Complexity Analysis

Preprocessing requires no preparatory data structure, giving $\Theta(1)$ initialisation time. The outer loop executes n - m + 1 times. In the worst case the inner loop executes m times per iteration, yielding $O(m \times (n - m + 1))$, which simplifies to $O(n \times m)$.

In the best case - when the first character of the pattern rarely appears in the text - the inner loop terminates after a single comparison at each position, giving O(n). For random text over a large alphabet a mismatch typically occurs within the first few characters, so the average case is close to O(n) in practice; the measurements in Section 3.3 confirm this, with Brute-Force running within a factor of 1.5 of KMP on the english, dna and binary corpora.

Auxiliary space is strictly O(1). Requiring no arrays, tables or hash state, Brute-Force has the smallest memory footprint of the four algorithms.

2.4.5. Theoretical Comparison of the Four Algorithms

Table 2.2 summarises the characteristics established in Sections 2.1 to 2.4.

Table 2.2: Theoretical comparison of the four algorithms

Algorithm	Preprocessing	Worst case	Average case	Auxiliary space	Key trade-off
Brute-Force	$\Theta(1)$	$O(n \times m)$	$O(n)$	$O(1)$	No memory overhead, but re-compares characters already seen and collapses on repetitive text.
KMP	$\Theta(m)$	$\Theta(n + m)$	$\Theta(n + m)$	$\Theta(m)$	Linear in every case; never backtracks the text pointer, so it works on streams. Cannot skip characters.
Z-function	—	$\Theta(n + m)$	$\Theta(n + m)$	$\Theta(n + m)$	Easiest to prove correct, but builds an array over the whole concatenation - the highest memory cost of the four.
Rabin-Karp	$\Theta(m)$	$O(n \times m)$	$O(n + m)$	$O(1)$	Constant space and the only one extending naturally to many patterns; large constant factor, and a poor modulus degrades the worst case.

No single algorithm dominates. Brute-Force is acceptable only when no preprocessing is affordable; KMP offers the strongest guarantee at the lowest memory cost among the linear algorithms; the Z-function trades memory for conceptual simplicity; and Rabin-Karp trades a large constant factor for constant space and multi-pattern capability. The appropriate choice therefore depends on measurable properties of the input - the text length n, the pattern length m, the alphabet size $|\Sigma|$ and the periodicity of the text - which is precisely what Chapter 3 sets out to measure.

CHAPTER 3: IMPLEMENTATION AND BENCHMARKING

3.1. Implementation Description

This section presents the technical architecture, programming environment, and source code implementation details for the four string matching algorithms studied in Chapter 2. Uniform implementation ensures objectivity when conducting empirical performance measurements and comparisons.

3.1.1. Programming Environment

The implementation is written in Python 3 and executed in a Google Colab notebook. An interpreted language exaggerates the constant factors of every algorithm relative to a compiled one. This does not distort the comparison between the four implementations, since all four pay the same penalty, but it does mean the absolute timings should not be read as the speed these algorithms would achieve in production. Section 3.3 quantifies the gap by including Python's built-in str.find, which is written in C, as a reference point.

3.1.2. Source Code Organisation

The whole implementation is contained in a single notebook, string_matching.ipynb, submitted as a separate file alongside this report. The notebook is organised into four stages that run in order: algorithm definitions, correctness verification, corpus and pattern preparation, and the benchmark together with its figures.

The four algorithms are implemented as the functions brute_force, kmp_search with its helper build_pi, z_search, and rabin_karp. Every one of them takes the text and the pattern as arguments and returns a pair: the list of matching start positions, and a dictionary of statistics. The statistics dictionary always contains the number of character comparisons; rabin_karp adds the number of hash agreements and the number of false positives.

This uniform return contract is what allows the benchmark loop to invoke all four algorithms through the same few lines of code, with no special case for any of them. It removes a subtle source of bias: if each algorithm were called through its own wrapper, differences in the wrappers would be measured alongside differences in the algorithms.

3.1.3. Implementation Details

Brute-Force is implemented with two nested loops over the indices i and j and no auxiliary structure, preserving its minimal character.

KMP is split into build_pi, which constructs the prefix array, and kmp_search, which scans the text using it. After a full match the search sets j to the prefix value rather than to zero, so that overlapping occurrences are reported.

The Z-function builds its array over the concatenation S = P + separator + T, using the byte \x00 as the separator because it occurs in none of the four corpora. The three cases of the Z-box are collapsed into a single min expression followed by an extension loop.

Rabin-Karp implements the polynomial rolling hash with base 256 and a modulus supplied as a parameter, so that the effect of hash collisions can be investigated by varying it. Section 3.3 sweeps the modulus across five values spanning seven orders of magnitude.

3.1.4. Correctness Verification

Benchmarking an incorrect implementation measures nothing, so correctness is established before any timing is performed. All four algorithms are run against a naive reference implementation built from Python string slicing, on 400 randomly generated binary strings with randomly chosen pattern lengths.

A binary alphabet is deliberately chosen for this test because it maximises the frequency of overlapping matches and of repeated characters, which is where an off-by-one error in a prefix array or a Z-box is most likely to surface. The four algorithms agree with the reference and with one another on all 400 cases. Only then does the benchmark begin.

3.1.5. The Quadratic Guard

Brute-Force is excluded from runs above n = 10^5. On the periodic corpus its O(n x m) behaviour would require hours for a single configuration, which is not compatible with repeating every measurement three times across four corpora and nine patterns.

The exclusions are not silently dropped. Each is written to skipped.csv with its corpus, size, pattern and reason, and 60 such rows were recorded. The file is itself a result: an algorithm that cannot complete within the time budget while three others finish the same configuration in under a second is stronger evidence of quadratic behaviour than any single timing would be. Section 3.3 reports these cells as not completed rather than leaving them blank.

3.2. Experimental Methodology and Dataset Preparation

3.2.1. Experimental Datasets

The benchmark uses four corpora chosen to span both alphabet size and text structure. Two are public datasets and two are generated by the group.

Table 3.1: The four benchmark corpora

Corpus	Length	|Σ|	Source
dna	5,534,032	4	Kaggle DNA sequence dataset, human sequences concatenated
english	11,744,905	96	Project Gutenberg prose obtained through the NLTK corpus
binary	11,744,905	2	Generated: uniformly random over {0, 1}
periodic	11,744,905	2	Generated: (ab) repeated to the required length

A fixed random seed of 42 governs every random generation and every pattern sample, so the same benchmark inputs can be reproduced exactly on any machine. The binary and periodic corpora are not public data and are described as generated wherever they appear; they exist to supply the extreme values of alphabet size and of text periodicity that the two real corpora do not provide.

The DNA file is cleaned before benchmarking: 530 occurrences of the character N are removed. N marks a base that could not be determined during sequencing. Keeping it would give the DNA corpus an alphabet of five symbols rather than the intended four, which would compromise

alphabet size as a controlled variable. One consequence of concatenating 4,380 separate sequences into a single string should also be noted: the joins are artificial and have no biological meaning. This is harmless for the present purpose, since only the statistical properties of the alphabet matter to a matching algorithm.

3.2.2. Pattern Types

Three kinds of pattern are used at each of three lengths, m = 5, 50 and 500.

Present patterns are cut directly from the corresponding text at a randomly chosen offset, so at least one occurrence is guaranteed. Three are sampled per length. This is the most realistic case and the one used for the main timing figures.

Absent patterns consist of a randomly generated body followed by the byte \x01, which lies outside the alphabet of every corpus. No occurrence is therefore possible. These separate the cost of searching from the cost of reporting occurrences: an algorithm that is slow on present patterns but fast on absent ones is spending its time handling matches rather than scanning.

Degenerate patterns consist of a single character repeated m times and were intended to create the worst case for Brute-Force. This design did not achieve its aim, and the failure is reported here rather than concealed. On a text such as ababab… a pattern aaa… mismatches at its second character, so the inner loop aborts immediately and no long partial match is ever built. The genuine worst case requires a pattern that almost matches at every alignment, which is what a present pattern sampled from periodic text produces. Section 3.3 accordingly uses present patterns for the worst-case analysis.

3.2.3. Controlled Experimental Conditions

Text length n is varied over 10^3, 10^4, 10^5 and 10^6. Every algorithm receives exactly the same byte range of the same corpus, the same pattern and the same input size, so that no difference in measured time can be attributed to a difference in input. All runs are executed on the same machine within a single interpreter session.

Each configuration is measured three times and both the mean and the minimum are recorded. The minimum is the more robust of the two against interference from background processes and from garbage collection; a large gap between the two indicates a measurement that should not be trusted. Timing uses a high-resolution performance counter rather than wall-clock time, so that adjustments to the system clock cannot affect a measurement.

Brute-Force is excluded above $n = 10^5$. Its quadratic behaviour on periodic text would make the run take hours, and the exclusions are recorded with their reason rather than silently dropped; Section 3.1 discusses this guard and the resulting evidence.

The source code is submitted as a separate file and is not reproduced in the report. Chapter 3 presents the experimental method, the figures, the results and the observations drawn from them.

3.3. Empirical Analysis and Performance Visualization

This section presents the measurements obtained from the benchmark described in Section 3.2. All timings are means over repeated runs on the identical byte range of the identical corpus, executed in a single interpreter session. Unless stated otherwise, figures use a pattern of length m = 50 sampled from the text itself, so that at least one match is guaranteed.

3.3.1. Execution Time against Text Length

The first view plots execution time against text length on logarithmic axes. On such axes a straight line of slope 1 indicates linear growth, and any steeper slope indicates a super-linear regime.

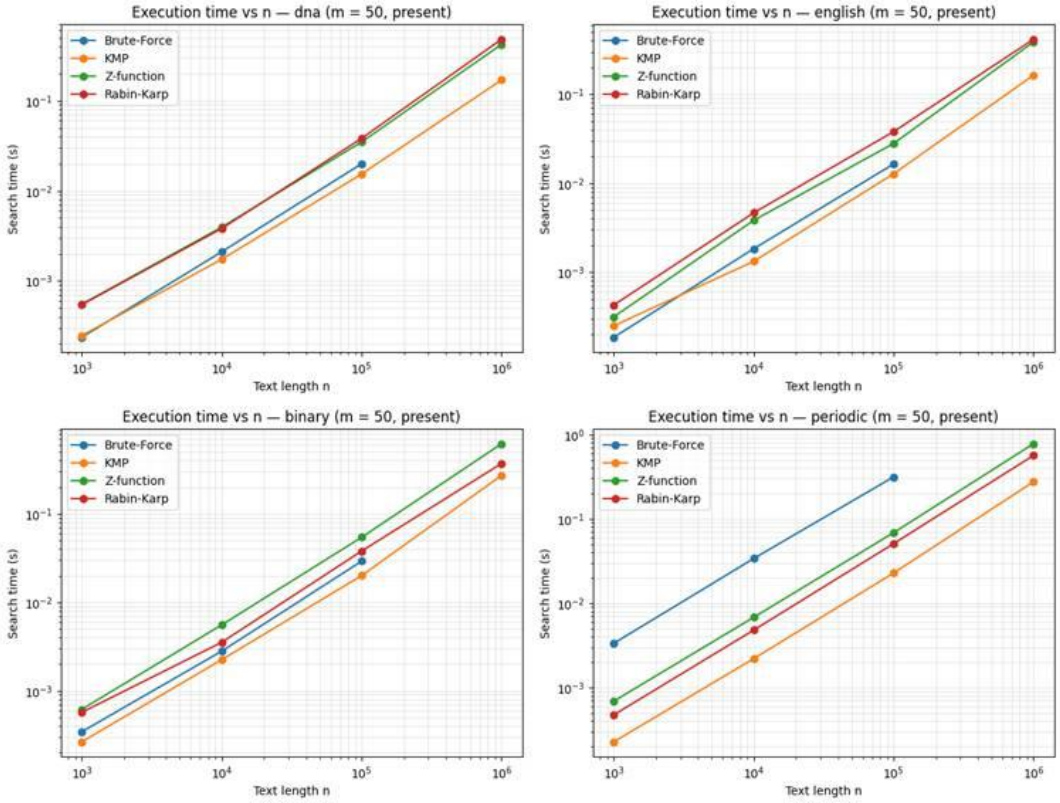


Figure 3.1: Execution time against text length n on log-log axes, one panel per corpus (m = 50)

Table 3.2 gives the underlying values at m = 50. Brute-Force was excluded above n = 10^5 because its quadratic behaviour on periodic text would have made the run infeasible; those exclusions are recorded in skipped.csv and are discussed in Section 3.1.

Table 3.2: Mean search time in seconds at m = 50, at the two largest text sizes. Smaller sizes are plotted in Figure 3.1

Corpus	n	Brute-Force	KMP	Z-function	Rabin-Karp
english	10^5	0.016471	0.012792	0.028043	0.038097
english	10^6	not completed	0.164264	0.388504	0.416544
dna	10^5	0.019933	0.015346	0.034779	0.038118
dna	10^6	not completed	0.169336	0.420221	0.478936
binary	10^5	0.029185	0.020033	0.054284	0.037940
binary	10^6	not completed	0.271642	0.619976	0.371041

periodic	10^5	0.314113	0.022822	0.068578	0.050832
periodic	10^6	not completed	0.275010	0.775255	0.564288

Three observations follow from the table. KMP is the fastest of the four on every corpus and at every size, reaching 0.164 s on english and 0.169 s on dna at $n = 10^6$. The Z-function trails it by a factor of 2.3 to 2.8 despite sharing the same $\Theta(n+m)$ bound, a constant-factor gap traceable to its array over the concatenated string of length $n + m + 1$ against KMP's array of length m. And each ten-fold increase in n multiplies the time by between 9.8 and 13.9, averaging 11.9: the growth is linear in shape, but consistently above the factor of ten, which is what would be expected once the working arrays outgrow processor cache at $n = 10^6$.

Brute-Force behaves differently by corpus. On english and dna it stays competitive at small n, because the first character of the pattern rarely matches and the inner loop aborts almost at once. On periodic it is an order of magnitude slower than KMP at every size, taking 0.314 s at $n = 10^5$ against 0.023 s.

3.3.2. Rabin-Karp against the Comparison-Based Algorithms

Rabin-Karp is the slowest or second slowest of the four here, running 1.4 to 2.8 times slower than KMP at $n = 10^6$ depending on the corpus. This does not contradict its $\Theta(n + m)$ bound: every window costs a multiplication, a subtraction and a modulo on arbitrary-precision integers, where KMP costs one character comparison. Its real advantages, constant auxiliary space and multi-pattern search, are not exercised by a single-pattern benchmark. It improves its position on two corpora, overtaking the Z-function at 0.371 s against 0.620 s on binary and at 0.564 s against 0.775 s on periodic. Rabin-Karp is in fact the only one of the four whose ranking changes with the corpus, which is the clearest single illustration of the claim that no algorithm dominates across all inputs.

3.3.3. The Effect of Alphabet Size

Fixing n and m and comparing across corpora isolates the influence of the alphabet, which ranges from $|\Sigma| = 2$ for the binary and periodic corpora, through $|\Sigma| = 4$ for DNA, to $|\Sigma| = 96$ for English prose.

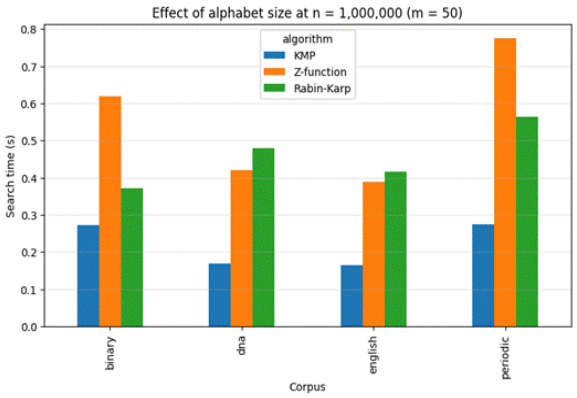


Figure 3.2: Search time by corpus at $n = 10^6$ and $m = 50$. Alphabet sizes are 2 for binary and periodic, 4 for dna and 96 for English

The effect is present but modest. KMP requires 0.272 s on binary against 0.164 s on english at n = 10^6, a spread of about 1.65, and the Z-function shows a comparable ratio at a higher absolute cost, 0.620 s against 0.389 s. The ordering is the same in both cases: the larger the alphabet, the faster the algorithm, because mismatches occur earlier. The growth order does not change, only the constant factor.

3.3.4. Character Comparisons and Pattern Length

Wall-clock time conflates algorithmic behaviour with machine speed and interpreter overhead. Counting character comparisons removes both, producing a measure that is reproducible on any hardware. Table 3.3 reports the ratio of Brute-Force comparisons to KMP comparisons at n = 10^5, for three pattern lengths.

Table 3.3: Ratio of Brute-Force to KMP character comparisons at n = 10^5, by pattern length

Corpus	m = 5	m = 50	m = 500	Time ratio at m = 500
english	1.00	1.00	1.00	1.31
dna	1.09	1.07	1.06	1.32
binary	1.40	1.39	1.43	1.49
periodic	3.00	25.48	248.01	139.32

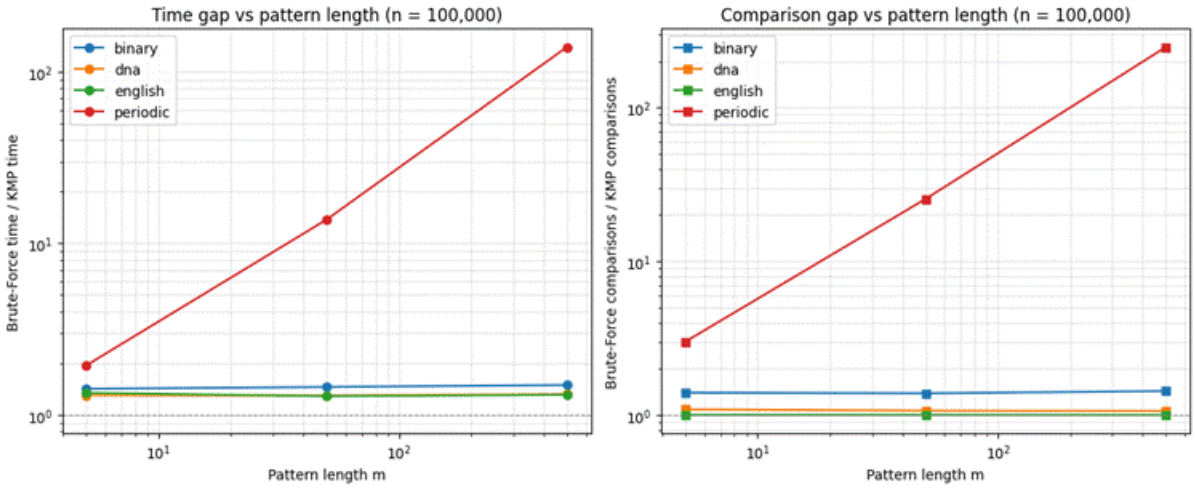


Figure 3.3: Ratio of Brute-Force to KMP against pattern length at n = 10^5. Left: wall-clock time. Right: character comparisons

On english, dna and binary the ratio is flat and close to unity, because on unstructured text the inner loop aborts after one or two characters and there is nothing for the π-array to save. On periodic it rises from 3.00 to 25.48 to 248.01 as m rises from 5 to 50 to 500 - approximately m/2, exactly what the difference between O(n×m) and O(n+m) predicts when the text has period 2. The Brute-Force penalty is therefore governed by pattern length, and appears only on text with internal periodicity.

One restriction applies to this metric. Comparison counts are meaningful only for the three comparison-based algorithms; Rabin-Karp performs modular arithmetic once per window and compares characters only after a hash agreement, so at $n = 10^6$ with a large modulus it may record fewer than a hundred comparisons while KMP records over a million, despite being two to three times slower in wall-clock terms. The two metrics are therefore reported separately, and wall-clock time is used whenever Rabin-Karp enters the comparison.

3.3.5. Rabin-Karp Modulus Sweep

Holding the corpus, the pattern and n fixed while varying q isolates the collision-versus-arithmetic trade-off described in Section 2.3.3.

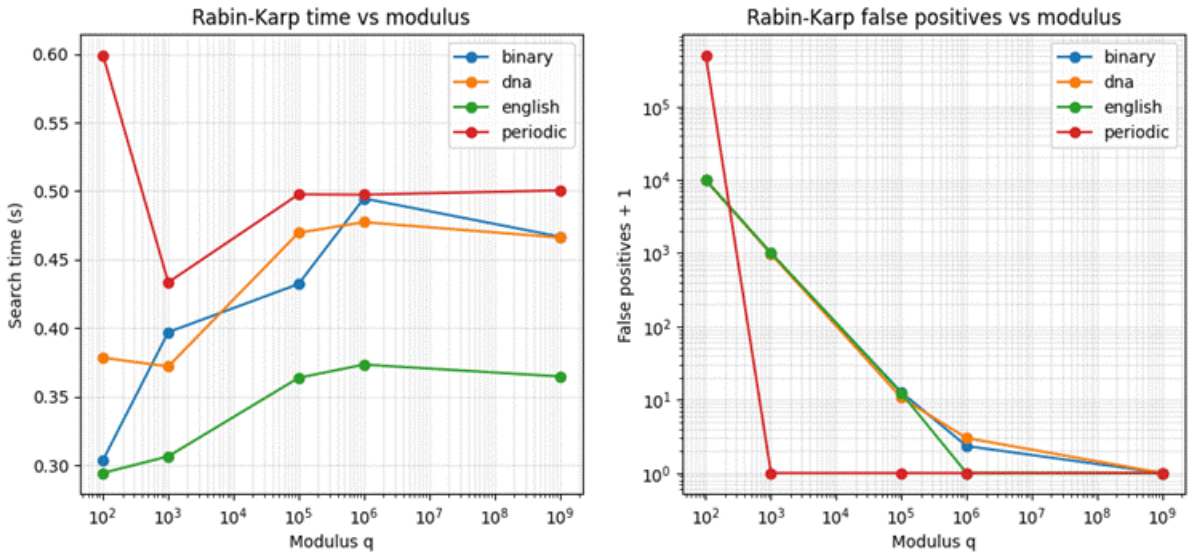


Figure 3.4: Rabin-Karp search time and false-positive count against modulus q, $n = 10^6$

Table 3.4: Rabin-Karp mean time in seconds and mean false-positive count against modulus, $n = 10^6$, $m = 50$

Modulus q	binary: time	binary: FP	english: time	english: FP
101	0.303561	9,879	0.294129	9,906
1,009	0.396940	989	0.306280	1,007
100,003	0.432101	12	0.363575	11
1,000,003	0.494560	1	0.373265	0
$10^9 + 7$	0.466709	0	0.364617	0

The false-positive count behaves exactly as theory predicts, falling by roughly a factor of ten for each ten-fold increase in q and reaching zero at the largest moduli. The dna corpus follows the same pattern, from 10,016 false positives at $q = 101$ down to none at $q = 10^9+7$.

The agreement is quantitative, not merely directional. For a hash that distributes uniformly the

expected number of spurious agreements is n/q, which at $n = 10^6$ predicts 9,901 at q = 101, 991 at q = 1,009 and 10 at q = 100,003. The measured counts were 9,879, 989 and 12 on binary; 9,906, 1,007 and 11 on english; and 10,016, 968 and 10 on dna - within a few percent of prediction on three corpora whose alphabets differ by a factor of twenty-four. This is direct evidence that the polynomial hash spreads values evenly over the modulus, independently of the text it is applied to.

The time column does not follow it. On every one of these corpora the smallest moduli produced the fastest searches, not the slowest: 0.304 s against 0.467 s on binary, and 0.294 s against 0.365 s on english. Roughly ten thousand false positives, each costing an O(m) verification, were cheaper than the arbitrary-precision arithmetic that a nine-digit modulus imposes on all one million windows.

The exception confirms the mechanism rather than contradicting it. On the periodic corpus at q = 101, 499,975 windows produced a hash agreement - very nearly half of all windows, and fifty times the 9,901 that uniformity would predict. The cause is structural rather than statistical: a text of period 2 contains only two distinct windows of any given length, so equal hashes arise from genuinely equal substrings rather than from collisions, and the assumption behind the n/q estimate does not hold. Search time there rose to 0.599 s, the slowest measurement in the sweep. Verification cost is paid only on agreement, whereas modular arithmetic is paid on every window; agreements therefore have to occupy a large fraction of all windows before they dominate.

3.3.6. Reference Baseline

Python's built-in str.find is implemented in C with a highly optimised hybrid algorithm. Including it establishes what fraction of the measured differences is attributable to algorithm choice and what fraction to implementation language.

Table 3.5: Single-run search time in seconds against the built-in implementation, m = 50

Corpus	str.find (C)	KMP (Python)	Z-function (Python)
english	0.000311	0.133543	0.317451
dna	0.001542	0.164261	0.366187
binary	0.008586	0.197259	0.704406
periodic	0.506042	0.468053	1.367547

On english the built-in function beats the KMP implementation by a factor of roughly 430, narrowing to 107 on dna and 23 on binary as the alphabet shrinks. On periodic the ordering reverses: str.find takes 0.506 s and the Python KMP implementation 0.468 s. A function written in C and optimised over decades loses to an interpreted loop, because on periodic input its skipping heuristics stop producing useful shifts while KMP's $\Theta(n+m)$ guarantee holds unconditionally.

3.4. Empirical Discussion

3.4.1 A smaller modulus can outperform a larger modulus in Rabin–Karp

A larger prime modulus was expected to perform better, since larger moduli produce fewer hash

collisions and therefore fewer character-by-character verifications. Three of the four corpora contradict this. On the binary corpus q = 101 required only 0.304 s despite generating about 9,879 false positives, while q = 10^9+7 required 0.467 s with none at all; the DNA and English corpora behave the same way.

The explanation is where each cost is paid. The rolling hash is recomputed for every sliding window, so the modulo cost accumulates across the whole scan, whereas verification is triggered only by a collision. A larger modulus pays off only when collisions are frequent enough to outweigh its arithmetic overhead. That happens on one corpus only: with q = 101 the periodic text produced roughly 499,975 colliding windows, close to half of all windows examined, and there the larger modulus was indeed faster.

3.4.2 The performance gap between Brute-Force and KMP is driven mainly by pattern length

Given O(n×m) for Brute-Force and O(n+m) for KMP, the gap between them might be expected to widen mainly as the text grows. The measurements point to pattern length instead. On the periodic corpus the ratio of Brute-Force comparisons to KMP comparisons rose from 3.00× at m = 5 to 25.48× at m = 50 and 248.01× at m = 500, roughly in proportion to m, while binary, DNA and English stayed between 1.0× and 1.4× at every pattern length.

Repetitive text is what produces this. Where most alignments share a long common prefix with the pattern, Brute-Force compares almost the entire pattern before finding a mismatch near its end, so the wasted work grows with m, while KMP reuses the matched prefix instead of restarting. Text length has no comparable effect, because at a fixed m both algorithms are linear in n and enlarging the text scales both by the same factor.

3.4.3 Built-in functions are not always the fastest choice

Python's str.find is written in optimised C and would normally be expected to beat any pure-Python implementation. On ordinary text it does: on the English corpus it completed the search in 0.00031 s against 0.134 s for KMP, roughly 430 times faster.

That advantage disappeared on the periodic corpus, where str.find took 0.506 s and KMP 0.468 s. The margin is small, but the reversal matters: KMP's prefix function gives a provable linear-time guarantee, whereas the built-in implementation is tuned for the average case and offers no such guarantee on highly repetitive input. As the input approaches its worst case, the algorithmic guarantee outweighs the implementation advantage.

3.4.4 KMP and the Z-function maintain linear scalability across all datasets

Both KMP and the Z-function grew linearly with text size on all four corpora, despite large differences in alphabet size and repetition. The binary and periodic corpora contain many repeated prefixes and the English corpus a much larger alphabet with few of them; these differences shifted the constant factors and produced small differences in execution time, but the growth rate stayed linear throughout.

This matches the Θ(n+m) bound. Both algorithms preprocess enough information for a matched character to be reused rather than compared again, so each character is handled a constant number of times whatever the input looks like. Neither alphabet size nor repetitiveness changes the asymptotic growth rate; they change only the constant.

3.4.5 Periodicity affects the three linear-time algorithms very differently

Comparing the periodic corpus against the binary corpus isolates repetition on its own, since the two share the same alphabet size, the same length and the same generator. At $n = 10^6$ that repetition cost KMP 1% (0.272 s to 0.275 s), the Z-function 25% (0.620 s to 0.775 s) and Rabin-Karp 52% (0.371 s to 0.564 s).

Sharing an asymptotic bound therefore does not imply sharing the behaviour. KMP is effectively immune because the π-array encodes precisely the periodic structure it meets. The Z-function pays more because its Z-box is repeatedly extended and truncated across a text in which every position matches the prefix. Rabin-Karp pays most because a periodic text contains roughly 500,000 genuine occurrences of the pattern, each triggering an O(m) verification that hashing cannot avoid.

CHAPTER 4: CONCLUSION

This report presented a comprehensive study of four exact string matching algorithms: Brute-Force, Knuth-Morris-Pratt (KMP), the Z-function, and Rabin-Karp. The study combined theoretical analysis with practical implementation and empirical benchmarking in order to evaluate how different algorithmic strategies behave under varying input characteristics.

From the theoretical perspective, the report demonstrated that KMP and the Z-function guarantee linear-time complexity, $\Theta(n + m)$, by avoiding redundant character comparisons through preprocessing techniques, while Rabin-Karp achieves expected linear performance by employing rolling hash computation. Brute-Force, although conceptually simple and memory-efficient, remains vulnerable to highly repetitive inputs because of its $O(n \times m)$ worst-case complexity.

The experimental evaluation confirmed most theoretical expectations while also revealing several important practical observations. KMP consistently achieved the best overall performance across all benchmark datasets due to its low preprocessing overhead and efficient memory usage. The Z-function exhibited the same asymptotic complexity but incurred a higher constant factor because it requires an auxiliary array over the entire concatenated string. Rabin-Karp demonstrated that practical performance depends heavily on the choice of modulus, showing that smaller moduli may outperform larger ones when collision rates remain sufficiently low. Furthermore, the experiments showed that the performance penalty of Brute-Force is primarily determined by pattern periodicity and pattern length rather than text length alone.

Overall, the results demonstrate that no single algorithm is universally optimal. The most appropriate choice depends on measurable properties of the input, including text length (n), pattern length (m), alphabet size ($|\Sigma|$), data periodicity, and application requirements such as memory availability or multi-pattern searching. This finding reinforces the importance of selecting algorithms according to practical operating conditions rather than relying solely on asymptotic complexity.

Future work could extend this study by evaluating additional string matching algorithms such as Boyer-Moore, Aho-Corasick, and suffix-based indexing methods, as well as comparing implementations across different programming languages and hardware platforms. Investigating approximate string matching and parallel implementations would also provide valuable directions for further research.

REFERENCES

[1] Aho, A. V., & Corasick, M. J. (1975). Efficient string matching: An aid to bibliographic search. Communications of the ACM, 18(6), 333-340.

[2] Altschul, S. F., Gish, W., Miller, W., Myers, E. W., & Lipman, D. J. (1990). Basic local alignment search tool. Journal of Molecular Biology, 215(3), 403-410.

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). Cambridge, MA: MIT Press.

[4] Gusfield, D. (1997). Algorithms on strings, trees, and sequences: Computer science and computational biology. Cambridge, England: Cambridge University Press.

[5] Jurafsky, D., & Martin, J. H. (2009). Speech and language processing (2nd ed.). Upper Saddle River, NJ: Pearson Prentice Hall.

[6] Karp, R. M., & Rabin, M. O. (1987). Efficient randomized pattern-matching algorithms. IBM Journal of Research and Development, 31(2), 249-260.

[7] Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. SIAM Journal on Computing, 6(2), 323-350.

[8] Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to information retrieval. Cambridge, England: Cambridge University Press.